📁 Node-RED – Sistema di Controllo OffGas

Questo documento descrive l'implementazione della logica di controllo del sistema **OffGas** tramite Node-RED.

Il documento copre:

1️⃣ architettura logica del flow
2️⃣ descrizione dei nodi Node-RED
3️⃣ algoritmo di predizione
4️⃣ logica decisionale (anomaly detection)
5️⃣ comunicazione MQTT con il Bridge.

Panoramica del ruolo di Node-RED

Nel sistema OffGas il **Bridge Python non prende più decisioni autonome**, ma svolge principalmente il ruolo di **gateway IoT** tra hardware e piattaforma software.

Il Bridge:

- legge il sensore MQ-2 da Arduino
- pubblica i dati via MQTT
- riceve comandi da Node-RED
- controlla la ventola via Bluetooth.

Nel codice del Bridge la telemetria viene pubblicata con:

```
payload = {

 "garage_id": "G1",

 "gas": data["gas"],

 "threshold": self.read_dataset['others_mean'] *
self.config.ANOMALY_FACTOR,
```

```
 "timestamp": data["timestamp"],

 "fan_state": self.fan_controller.fan_state

}
```

self.mqtt.publish_telemetry(payload)

Node-RED riceve questi dati e svolge tre funzioni principali:

1. analisi dei dati
2. predizione del gas
3. decisione sullo stato della ventola.

Architettura logica del Flow Node-RED

Il flusso logico implementato in Node-RED è il seguente:

Bridge MQTT telemetry

↓

mqtt in node

↓

PredictionNextValue (function)

↓

AnomalyDetection (function)

↓

mqtt out → Bridge

In parallelo sono presenti nodi per il **controllo manuale** della ventola.

---

1) Nodo MQTT IN – Ricezione telemetria

Il nodo MQTT IN riceve la telemetria pubblicata dal Bridge.

Configurazione:

```
topic: garages/G1/telemetry

datatype: json
```

Questo nodo riceve messaggi JSON con la struttura:

```
{

 "garage_id": "G1",

 "gas": 210,

 "threshold": 117.7,

 "fan_state": true,

 "timestamp": "..."

}
```

Nel flow Node-RED questo nodo è definito come:

```
"type": "mqtt in",

"topic": "garages/G1/telemetry"
```

Il nodo genera un oggetto msg.payload contenente il JSON.

Questo payload viene poi inviato a due nodi:

-PredictionNextValue

-debug

Il nodo debug è utilizzato per monitorare i dati ricevuti.

---

2) Nodo Function – PredictionNextValue

Questo nodo implementa **l'algoritmo di predizione del gas**.

L'obiettivo è stimare il valore del gas tra circa **30 secondi nel futuro**.

Concetto dell'algoritmo

Il metodo utilizzato è:

media mobile + velocità di crescita

Il nodo mantiene uno storico degli ultimi valori del gas.

Nel codice:

```
var history = context.get("gas_history") || [];
```

Node-RED salva i valori nel **context del nodo**, che funziona come una memoria persistente tra le esecuzioni.

Ogni nuovo valore viene aggiunto allo storico:

```
history.push(gas);
```

Lo storico è limitato a 20 valori:

```
if (history.length > 20) {

    history.shift();

}
```

Calcolo della media mobile

La media mobile rappresenta il livello medio del gas negli ultimi campioni.

```
var sum = 0;

for (var i = 0; i < history.length; i++) {

    sum += history[i];

}

var movingAvg = sum / history.length;
```

Questo permette di **ridurre il rumore del sensore MQ-2**.

Calcolo della velocità di crescita

La crescita del gas viene stimata confrontando la media corrente con quella precedente.

```
var growthRate = movingAvg - previousAvg;
```

Se growthRate è positivo significa che il gas sta aumentando.

Predizione del valore futuro

Il valore futuro viene stimato con:

```
var predictedGas = gas + (growthRate * 150);
```

Il numero **150** rappresenta l'orizzonte temporale della predizione.

Nel Bridge il ciclo principale è:

```
time.sleep(0.2)
```

Quindi:

```
0.2 secondi per ciclo

150 cicli ≈ 30 secondi
```

Valori aggiunti al payload

Il nodo aggiunge al payload:

```
msg.payload.predicted_gas
```

```
msg.payload.predicted_crossing
```

Esempio:

```
{
 "gas": 200,
 "threshold": 117.7,
 "predicted_gas": 245,
 "predicted_crossing": true
}
```

Il flag predicted_crossing indica se il gas **supererà la soglia nel futuro**.

---

3) Nodo Function – AnomalyDetection

Questo nodo implementa la **logica decisionale del sistema**.

Il codice del nodo è:

```
var gas = msg.payload.gas;

var threshold = msg.payload.threshold;

msg.payload = {

    mode: "STD",

    anomaly: gas > threshold,

    predicted_crossing: msg.payload.predicted_crossing

};

return msg;
```

Questo nodo genera il comando che verrà inviato al Bridge.

Logica decisionale

La decisione è basata su due condizioni:

1. gas sopra soglia
2. gas previsto sopra soglia.

La soglia viene calcolata nel Bridge:

```
threshold = others_mean * ANOMALY_FACTOR
```

Nel codice Node-RED:

```
anomaly = gas > threshold
```

Se:

```
anomaly = true
```

significa che il gas è già sopra soglia.

Se invece:

```
predicted_crossing = true
```

significa che la soglia verrà superata nel futuro.

Il Bridge utilizza queste informazioni nel metodo on_message.

Nel codice Python:

```
if command["mode"] == "STD":

    if command["anomaly"] or command["predicted_crossing"]:

        self.fan_controller.force_on()
```

Payload inviato al Bridge

Il JSON finale inviato al Bridge è:

```
{
 "mode": "STD",
 "anomaly": true,
 "predicted_crossing": false
}
```

---

4) Nodo MQTT OUT – Invio comando al Bridge

Il nodo MQTT OUT pubblica il comando sul topic:

```
garages/G1/cmd
```

Configurazione:

```
"type": "mqtt out"
"topic": "garages/G1/cmd"
```

Questo messaggio viene ricevuto dal Bridge nel metodo:

```
def on_message(self, client, userdata, msg):
```

Il Bridge decodifica il JSON con:

```
command = json.loads(raw)
```

---

5) Controllo manuale della ventola

Nel flow Node-RED sono presenti tre nodi **inject** che permettono il controllo manuale.

Questi nodi simulano pulsanti della dashboard.

Comando FAN_ON

Nodo inject:

```
payload: "FAN_ON"
```

Nodo MQTT OUT:

```
topic: garages/G1/cmd
```

Il Bridge interpreta il comando con:

```
if command in ("FAN_ON", "ON"):
```

Comando FAN_OFF

Nodo inject:

```
payload: "FAN_OFF"
```

Il Bridge esegue:

```
self.fan_controller.force_off()
```

Comando AUTO

Nodo inject:

```
payload: "AUTO"
```

Il Bridge ritorna alla modalità automatica:

```
self.fan_controller.set_auto_mode()
```

Modalità operative del sistema

Ricapitolando, il sistema può funzionare in due modalità.

Modalità automatica (STD)

Node-RED decide lo stato della ventola.

La decisione è basata su:

```
gas > threshold
```

OR

```
predicted_crossing
```

Modalità manuale

Decido se forzare l'accensione della ventola. Node-RED invia:

```
FAN_ON
```

```
FAN_OFF
```

Il Bridge attiva manual_override.

Nel codice:

```
self.fan_controller.manual_override = True
```

In questa modalità la logica automatica viene ignorata.

---

6) Sistema di notifiche Telegram

Per migliorare il monitoraggio del sistema OffGas è stato integrato un **bot Telegram** in Node-RED.
Questo permette di inviare notifiche automatiche all'utente quando si verificano eventi rilevanti nel sistema, come:

- attivazione automatica della ventola per superamento soglia gas
- ritorno a condizioni normali
- attivazione manuale della ventola
- ritorno alla modalità automatica.

Le notifiche Telegram permettono quindi di monitorare il sistema **anche da remoto**, senza dover accedere alla dashboard Node-RED.

Architettura dei nodi Telegram

Nel flow Node-RED i nodi coinvolti nella gestione delle notifiche sono:

AnomalyDetection / Inject
↓
TelegramAlert (function)
↓
telegram sender
↓
Telegram Bot

Il nodo centrale della logica è **TelegramAlert**, che prepara il messaggio da inviare al bot.

Nodo Function – TelegramAlert

Il nodo TelegramAlert ha il compito di:

1. analizzare i messaggi provenienti dal sistema
2. determinare se è necessario inviare una notifica
3. formattare il messaggio Telegram.

Nel flow Node-RED questo nodo è definito come:

"type": "function",
"name": "TelegramAlert"

Il nodo riceve due tipi di messaggi:

- comandi manuali (stringhe)
- messaggi automatici provenienti dai sensori (oggetti JSON).

Gestione dello stato della ventola

Per evitare l'invio di notifiche duplicate, il nodo mantiene lo stato della ventola nel **context Node-RED**.

Nel codice:

```
let fanStatus = context.get("fan_status") || false;
```

Questo valore indica se la ventola è attualmente:

false → ventola spenta
true → ventola accesa

Il nodo utilizza inoltre un'altra variabile di contesto:

```
context.set("manual_mode", true);
```

che indica se il sistema è in **modalità manuale**.

Formattazione del messaggio Telegram

Il messaggio viene costruito tramite una funzione dedicata:

```
function formatTelegram(text) {
    return {
        chatId: CHAT_ID,
        type: "message",
        content: text
    };
}
```

Questa funzione genera il payload richiesto dal nodo **telegram sender**.

Il messaggio contiene:

- chatId → identificativo della chat Telegram
- type → tipo di messaggio
- content → testo da inviare.

Notifiche per comandi manuali

Il nodo intercetta i comandi manuali inviati da Node-RED:

```
FAN_ON
FAN_OFF
AUTO
```

Quando viene ricevuto il comando:

```
FAN_ON
```

il nodo invia una notifica:

*"**COMANDO MANUALE**
Ventola ACCESA.
Sensori ignorati fino al comando AUTO."*

Nel codice:

```
if (msg.payload === "FAN_ON") {
    context.set("manual_mode", true);
}
```

Questo segnala che la modalità automatica è stata disabilitata.

Notifica di spegnimento manuale

Se viene inviato il comando:

```
FAN_OFF
```

il nodo invia una notifica:

*COMANDO MANUALE
Ventola SPENTA.
Sensori ignorati fino al comando AUTO.*

Nel codice:

```
if (msg.payload === "FAN_OFF") {
    context.set("manual_mode", true);
}
```

Notifica ritorno modalità automatica

Quando viene inviato il comando:

```
AUTO
```

il nodo comunica il ritorno al controllo automatico:

MODALITÀ AUTO
Il sensore torna a gestire la ventola.

Nel codice:

```
if (msg.payload === "AUTO") {
    context.set("manual_mode", false);
}
```

Notifiche automatiche dai sensori

Il nodo TelegramAlert analizza anche i messaggi generati dalla logica automatica del sistema.

Il payload ricevuto da Node-RED ha la struttura:

```
{
 "mode": "STD",
 "anomaly": true,
 "predicted_crossing": true,
 "predicted_gas": 240
}
```

Se viene rilevata una condizione di anomalia:

```
anomaly = true
```

il nodo invia una notifica Telegram.

Nel codice:

```
if (anomaly && !fanStatus)
```

Messaggio inviato:

ALLARME GAS (G1)
Gas previsto: 240
Soglia superata. Ventola attivata.

Notifica rientro dei valori

Quando i valori del gas tornano sotto soglia:

```
anomaly = false
```

il nodo invia una seconda notifica:

GAS RIENTRATO (G1)
Valori nella norma. Ventola disattivata.

Nel codice:

```
if (!anomaly && fanStatus)
```

Nodo Telegram Sender

Il nodo telegram sender è responsabile dell'invio effettivo del messaggio.

Nel flow Node-RED è definito come:

"type": "telegram sender"

Questo nodo utilizza la libreria Node-RED:

node-red-contrib-telegrambot

Il bot configurato è:

@OffGasMonitor1Bot

Il nodo invia il messaggio al server Telegram utilizzando l'API ufficiale del bot.

Debug dei messaggi Telegram

Nel flow è presente anche un nodo debug collegato al nodo Telegram.

Questo nodo permette di verificare nel pannello debug di Node-RED:

- il contenuto dei messaggi inviati
- eventuali errori di invio.

Ruolo del sistema Telegram nel progetto

L'integrazione di Telegram permette di trasformare il sistema OffGas in un sistema di monitoraggio remoto.

L'utente riceve notifiche in tempo reale quando:

- viene rilevata una condizione pericolosa
- la ventola viene attivata
- il sistema ritorna in condizioni normali
- viene attivato un comando manuale.

Flusso completo del sistema

Il funzionamento del sistema OffGas è il seguente.

1. Arduino misura il gas
2. Arduino invia il dato al Bridge
3. Bridge pubblica telemetria MQTT
4. Node-RED riceve i dati
5. Node-RED esegue predizione
6. Node-RED calcola anomaly
7. Node-RED invia comando al Bridge e notifica a telegram
8. Bridge controlla la ventola.

Vantaggi dell'architettura

Separare la logica in Node-RED offre diversi vantaggi.

Edge Gateway semplice

Il Bridge rimane un componente leggero.

Logica modificabile

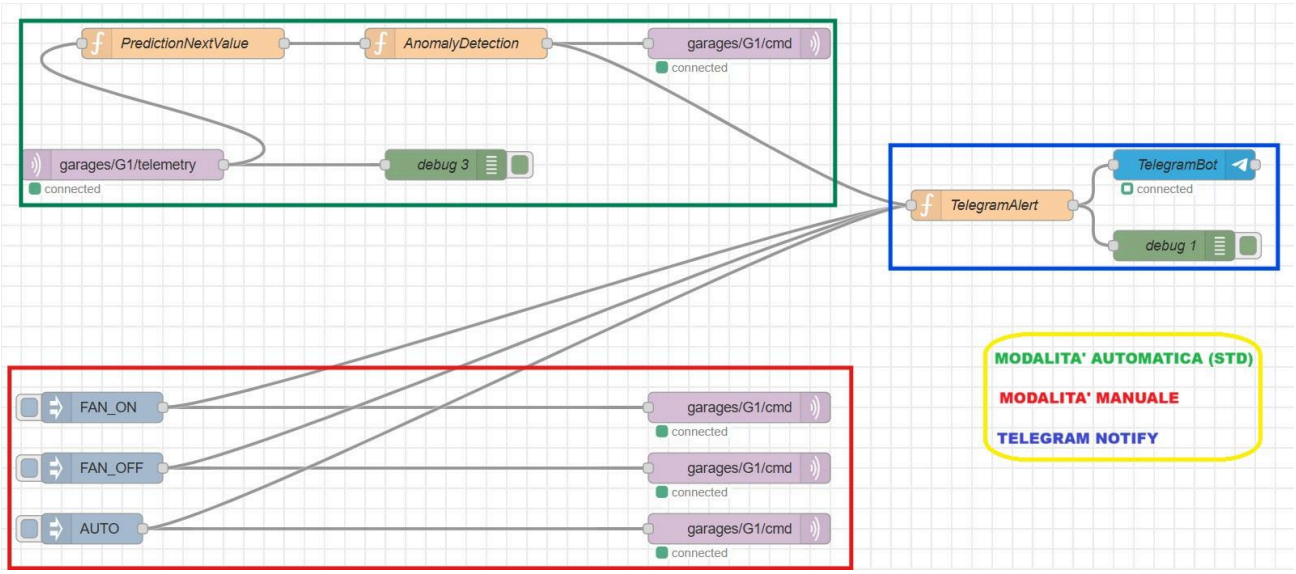
La logica può essere cambiata in Node-RED senza modificare il codice Python.

Predizione avanzata

Node-RED permette di implementare facilmente algoritmi di predizione.

Integrazione dashboard

Node-RED può mostrare:

- grafici storici
- stato ventola
- notifiche.

FLOW NODE_RED